

mGuard: A Secure Real-time Data Distribution System with Fine-Grained Access Control for mHealth Research

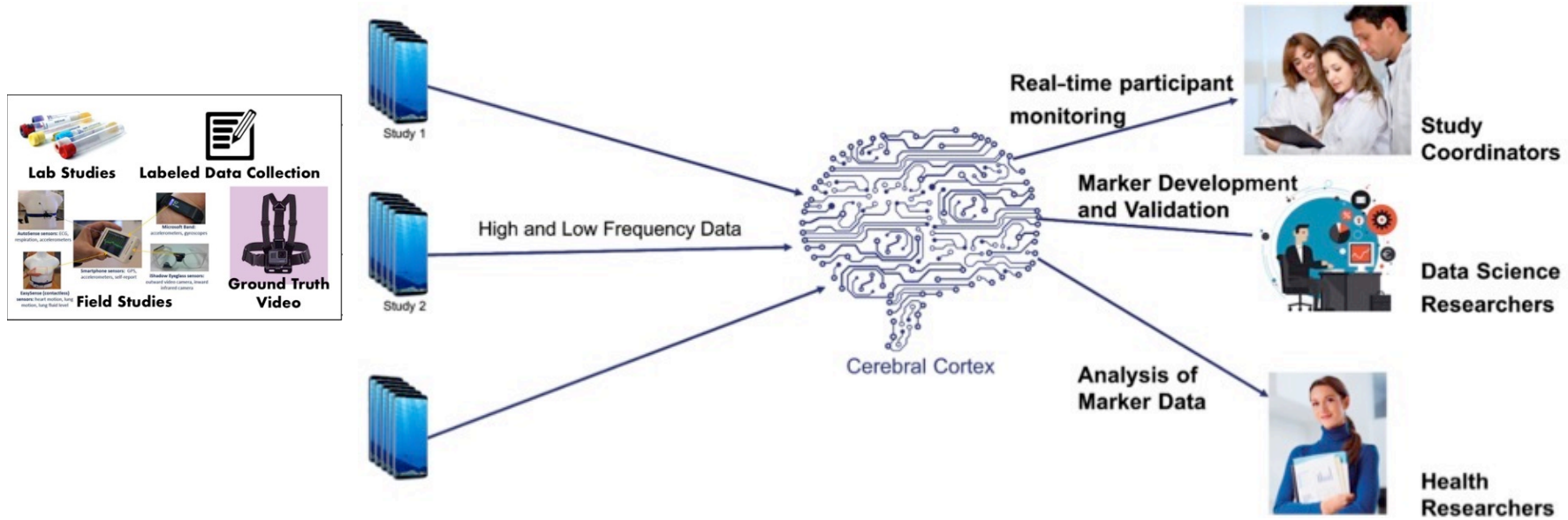
Lan Wang, Saurab Dulal, Chaudhry Nasir Ali, Adam
Robert Thieme, Santosh Kumar, U. Memphis
Siqi Liu, Lixia Zhang, UCLA

NDNComm 2021

This project is supported by NSF CC* grant 2019085.

mHealth Data Collection and Distribution Process (MD2K Center at U. Memphis)

- 2,200+ participants, 100,000+ participant-days
- 4.7 trillion data points, 300TB total data size



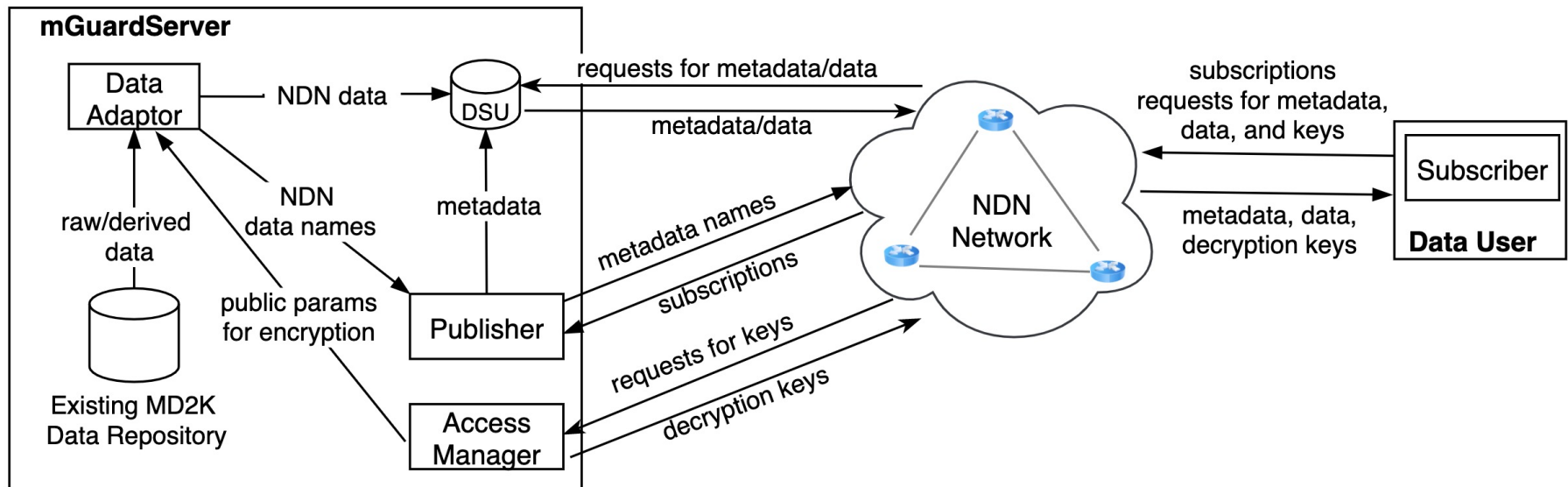
Data collected by embedded and wearable sensors enables research on health and wellness issues.

Challenges and Goal



- Existing mobile health (mHealth) data infrastructure lacks support for
 - Fine-grained access control
 - Real-time data distribution
- mGuard: apply NDN's name-based data-centric approaches (Sync [1] and NAC [2]) to solve the above problems
- a collaboration with MD2K

mGuard System Overview



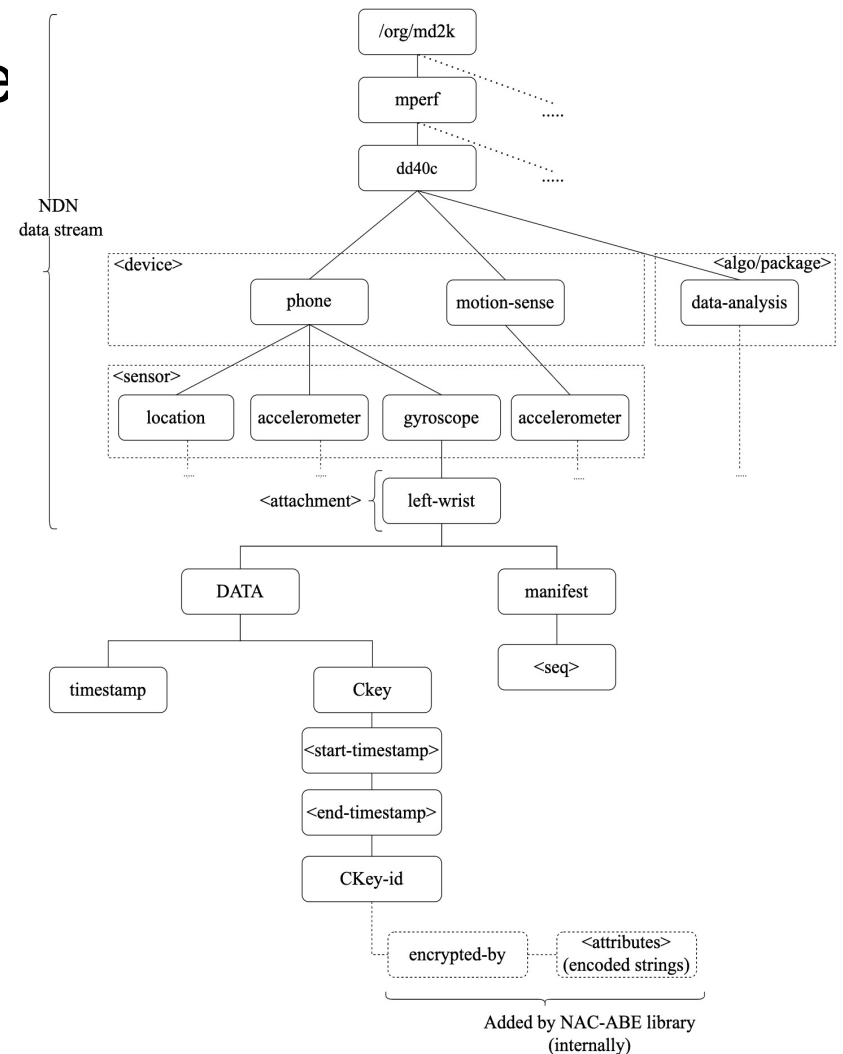
- **Data Adaptor** converts *mHealth* data to NDN data objects, encrypt them, and store them in an **NDN Repo**.
- **Publisher** advertises information about newly published data using **Pub-Sub API**.
- **Subscriber** subscribes to data streams using **Pub-Sub API** and fetches data from Repo.
- **Access Manager** generates public params to encrypt data and keys to decrypt data (based on **NAC-ABE**).

Project Progress in Year 1

- developed overall mGuard system design;
- designed NDN naming scheme for MD2K's mHealth data;
- developed access control policy specification and policy parser;
- extended NAC-ABE library to support KP-ABE (key policy attribute based encryption);
- designed pub-sub API using PSync and NAC-ABE for publishing and subscribing to data streams.

mGuard Naming Scheme

- Raw data stream:
/`<root>`/`<study>`/`<participant>`/`<devicename>`/`<sensor>`/`<attachment>`*
- Derived data stream:
/`<root>`/`<study>`/`<participant>`/`<algo/package>`
- Data: /`<stream-prefix>`/DATA/`<timestamp>`
- Content key: /`<stream-prefix>`/Ckey/...
- Manifest: /`<stream-prefix>`/manifest/`<seq>`



Access Control

- Goal: define and enforce fine-grained rights to access the data.
 - Support different types of users, e.g., data producers, study coordinators, and data administrators.
 - Give each user minimal access rights based on the terms of use.
 - Do not require expert domain knowledge to define a policy.
- Approach: leverage NAC-ABE [2] to (1) encrypt data with content key; (2) encrypt content key with attributes; (3) generate and distribute decryption keys to users.

Fine-Grained Access Control Policies

- Data owner defines a set of attributes based on the policies to support.
- Data owner specifies policies containing the set of attributes that a user is associated with.

```
policy-id          int
requester-names    requesters' NDN names
attribute-filters
{
    allow
    {
        attribute 1
        attribute 2
        ...
    }
    deny
    {
        attribute 1
        attribute 2
        ...
    }
}
```

- Example: [Alice at UCLA](#) can access [MD2K data](#) produced by the [mperf](#) study participant [dd40c](#) when the participant was at [home](#) or at [work](#), but Alice cannot obtain the participant's phone [GPS locations](#).

```
policy-id          3
requester-names    /edu/ucla/alice
attribute-filters
{
    allow
    {
        /org/md2k/mperf/dd40c/
        /attribute/location/home
        /attribute/location/work
    }
    deny
    {
        /org/md2k/mperf/dd40c/phone/gps-coordinate
    }
}
```

Access Control Policy Support

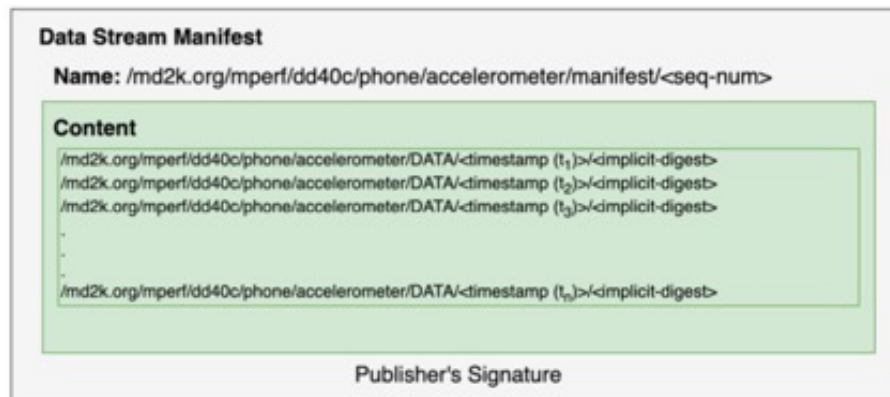
- Policy parser converts each policy to an expression, e.g.,
`(/attribute/location/home or /attribute/location/work) and (/attribute/date>=20210901)`
 - User's decryption key encodes the expression.
- NAC-ABE library [3]
 - access manager: automated key management
 - attribute-based encryption and decryption
 - Original implementation supports ciphertext-policy ABE. We extended it to support key-policy ABE (policy is encoded in decryption key).

Real-Time Data Distribution using Pub-Sub

- Pub-Sub API
 - allow users to subscribe to data streams as permitted by their access policies.
 - guarantee the reception of the latest and verified data in each subscribed stream.
- The pub-sub library uses PSync [4] to discover and subscribe to data streams, e.g., /org/md2k/mOral20/gyro, and use NAC-ABE to encrypt/decrypt data.

Problem: Lots of Small Data Objects

- Signing, verifying, and sync'ing every data object incurs high overhead.
- Solution: Manifest
 - Publisher produces a series of Manifests for each data stream, with the name `<stream-prefix>/manifest/<seq-num>`.
 - A manifest contains the full name (including the implicit digest) of new data objects.



- Data objects are not signed, but their implicit digests are verified.
- Sync manifests between publishers and subscribers.

Open Issues and Plan

- Lessons learned
 - Lots of time spent understanding access control requirements and NAC-ABE
 - Policy specification should be attribute based.
- Open issues
 - Lack of negative (i.e., NOT) operator support in NAC-ABE → inefficient to support “denying” attributes
 - Scalability of NAC-ABE approach: large number of attributes when policies are very fine-grained
 - Need a reliable Repo implementation (reliable high-speed data insertion)
- Year 2 plan
 - Finish system implementation and testing
 - Deploy mGuard on the NDN testbed: connect MD2K server (publisher) and users (subscribers) to the testbed

Questions?

References

- [1] P. Moll, W. Shang, Y. Yu, A. Afanasyev, and L. Zhang. A Survey of Distributed Dataset Synchronization in Named Data Networking. Technical Report NDN-0053, Revision 2, Named Data Networking, May 2021.
- [2] Z. Zhang, Y. Yu, S. K. Ramani, A. Afanasyev, and L. Zhang. NAC: Automating access control via Named Data. In Proceedings of IEEE MILCOM 2018, 2018.
- [3] NAC-ABE Library GitHub Site. <https://github.com/UCLA-IRL/NAC-ABE>.
- [4] M. Zhang, V. Lehman, and L. Wang. Scalable name-based data synchronization for named data networking. In IEEE INFOCOM 2017 - IEEE Conference on Computer Communications, 2017.